Then you add a label, text area, and validation tag helper to the Create form bound to the property via the `asp-for` attribute.

Listing 13.15 Adding a label and text area for the `Description` property

```
<div class="form-group">
    <label asp-for="Description" class="control-label"></label>
    <textarea asp-for="Description" class="form-control"></textarea>
    <span asp-validation-for="Description" class="text-danger"></span>
</div>
```

You install the AngleSharp library using your preferred package management approach. It is simply called AngleSharp, so the command is easy to intuit: `install-package AngleSharp` or `dotnet add package Anglesharp`. Then you add a using directive in the `CreateModel` class to bring the library into scope:

```
using AngleSharp;
```

Finally, you modify the `OnPostAsync` method to use AngleSharp. The first two lines of the newly added code are boilerplate code that prepares AngleSharp for consuming HTML, while the third line parses the `Description` property and returns an `IDocument`, which serves as an entry point into the parsed HTML. The `IDocument` adheres to the W3C DOM API, so methods like `QuerySelectorAll` are supported. In this example, you use it to query the HTML fragment for script elements. If there are any, you reject the submission, adding an entry to the `ModelState` errors collection.

Listing 13.16 Using AngleSharp to validate posted HTML before saving it

```
public async Task<IActionResult> OnPostAsync()
{
    var config = Configuration.Default.WithDefaultLoader();     ❶
    var context = BrowsingContext.New(config);                  ❶
    var document = await context.OpenAsync(req =>
    ➥ req.Content(Description));                               ❷
    if (document.QuerySelectorAll("script")
    ➥ .Any())                                                  ❸
    {                                                           ❸
        ModelState.AddModelError("Description",                 ❸
    ➥ "You must not include script tags");                     ❸
    }                                                           ❸
    if (ModelState.IsValid)
    {
        TempData["Name"] = Name;
        Photo = $"{Name.ToLower().Replace(
        ➥ " ", "-")}{Path.GetExtension(Upload.FileName)}";
```